

CTSE and dual numbers in CR form

Four worked examples for timestep sensitivities

Background: what are we trying to compute?

The code in this repository advances a thermal model through a timestep t . We want both the final temperatures and their derivatives with respect to t : how quickly would those temperatures change if the timestep changed, while the starting temperatures, model matrices, input, and forcing stayed fixed?

This note explains the calculation using four examples: the scalar function $f(x) = \sin(x)$ and the matrix function $\mathcal{F}(X) = \sin(X)$ evaluated by CTSE, then the same two functions evaluated using dual numbers in a real Cauchy–Riemann (CR) representation. We keep only **first derivatives**.

Automatic differentiation (AD). AD carries a value and its derivative through a calculation, applying the chain rule at each operation. For a sine operation $y = \sin(x)$, forward-mode AD carries the input derivative $\dot{x}$ to the output derivative $\dot{y} = \cos(x)\dot{x}$. Seeding $\dot{x} = 1$ gives $dy/dx = \cos(x)$. Dual numbers provide arithmetic that does this automatically; their CR form lets us perform that arithmetic with ordinary real matrices.

Why not just use finite differences? A centered finite difference estimates the derivative using two evaluations:

$$f'(t) \approx \frac{f(t+h) - f(t-h)}{2h}.$$

Its truncation error decreases like h^2 , but sufficiently small h makes the numerator a subtraction of nearly equal numbers. Rounding errors can then dominate. This motivates methods that carry derivative information within an evaluation.

Complex Taylor series expansion (CTSE). For an analytic function that is real on real inputs, replace t by $t + ih$, where $i^2 = -1$. Taylor expansion gives

$$f(t + ih) = f(t) + ihf'(t) - \frac{h^2}{2}f''(t) - \frac{ih^3}{6}f'''(t) + \dots.$$

Consequently,

$$f'(t) \approx \frac{\operatorname{Im} f(t + ih)}{h}.$$

This estimate has error of order h^2 without subtracting nearby function values. CTSE is a numerical approximation; dual-number AD propagates the first derivative without a finite step. Both still incur floating-point error.

How to read the examples. Each follows *seed the input* $\rightarrow$ *evaluate* $\rightarrow$ *extract*. The formulas supply a reference that can be checked by hand. The Julia snippets compute the derivative from the seeded evaluation, without using that reference.

1 CTSE: a scalar input

Let

$$f(x) = \sin(x), \quad x_0 = 1.$$

The reference value and derivative are $f(1) = \sin 1$ and $f'(1) = \cos 1$ (angles in radians).

1. Seed the input. Choose a small real step $h > 0$ and set $\tilde{x} = 1 + ih$. The imaginary part tags the change in the input.

2. Evaluate the sine. The complex sine identity is

$$\sin(a + ib) = \sin(a) \cosh(b) + i \cos(a) \sinh(b).$$

Applying it to the seeded input gives

$$\tilde{y} = \sin(\tilde{x}) = \sin(1) \cosh(h) + i \cos(1) \sinh(h).$$

Here $\sinh$ and $\cosh$ are the hyperbolic sine and cosine. This identity explains the result; it does not need to be implemented explicitly. If the function's code supports complex inputs throughout and preserves the imaginary part, the complex arithmetic propagates automatically. For this example, simply evaluate `sin(1 + im*h)`.

3. Extract the value and derivative.

$$\begin{aligned} f(1) &\approx \operatorname{Re} \tilde{y} = \sin(1) \cosh(h), \\ f'(1) &\approx \frac{\operatorname{Im} \tilde{y}}{h} = \cos(1) \frac{\sinh(h)}{h}. \end{aligned}$$

As h decreases, the extracted derivative approaches $\cos 1$. More precisely, it is $\cos(1) + \frac{1}{6} \cos(1)h^2 + O(h^4)$.

Julia calculation (`examples/ctse_scalar.jl`).

```
x0, h = 1.0, 1e-20
f(x) = sin(x)
y = f(x0 + im*h)
value = real(y)
derivative = imag(y)/h
@show value derivative
```

Compare with the analytical result. Direct differentiation gives $f'(x) = \cos(x)$. At $x = 1$, the comparison is

Quantity	CTSE	Analytical
$f(1) = \sin(1)$	0.841470984808	0.841470984808
$f'(1) = \cos(1)$	0.540302305868	0.540302305868

Both computed entries equal the analytical values rounded to double precision (references evaluated at 256-bit precision). The real part approximates the value with an $O(h^2)$ error too. An ordinary real evaluation supplies the nominal value if needed independently of h .

What the code must preserve. The imaginary part must survive every operation. Taking an absolute value, conjugating an input, or converting an intermediate result to real arithmetic can invalidate CTSE. The step 10^{-20} works for these examples; accuracy also depends on the numerical routine used to evaluate the function.

2 CTSE: a matrix input

Consider the matrix function $\mathcal{F}(X) = \sin(X)$, where

$$\sin(X) = X - \frac{X^3}{3!} + \frac{X^5}{5!} - \dots$$

This is a *matrix sine*: the powers use matrix multiplication. Let the input matrix depend on the scalar t through $X(t) = tH$, where H is a fixed matrix. In this example,

$$H = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}, \quad X(t) = tH, \quad F(t) = \mathcal{F}(X(t)), \quad t_0 = 1.$$

Thus we seek the derivative of $\sin(X(t))$ with respect to t . The input has the same timestep dependence as in the thermal code; here we apply sine to illustrate the differentiation method.

1. Seed the matrix input. Replace $t = 1$ by $t = 1 + ih$ in $X(t) = tH$. This gives the complex matrix input

$$\tilde{X} = (1 + ih)H = \begin{bmatrix} 0 & 1 + ih \\ -1 - ih & 0 \end{bmatrix}.$$

The upper-right entry changes from 1 to $1 + ih$, and the lower-left entry changes from -1 to $-1 - ih$. The diagonal entries remain zero.

2. Evaluate the matrix sine. Substitute $X = tH$ into the sine series. Since t is a scalar, $(tH)^k = t^k H^k$. Also, $H^2 = -I$, so $H^3 = -H$, $H^5 = H$, and the odd powers continue to alternate in sign. Thus

$$\begin{aligned} \sin(tH) &= tH - \frac{(tH)^3}{3!} + \frac{(tH)^5}{5!} - \dots \\ &= tH - \frac{t^3(-H)}{3!} + \frac{t^5 H}{5!} - \dots \\ &= H \left(t + \frac{t^3}{3!} + \frac{t^5}{5!} + \dots \right) = H \sinh(t). \end{aligned}$$

Substituting $t = 1 + ih$, use $\sinh(1 + ih) = \sinh(1) \cos(h) + i \cosh(1) \sin(h)$ to obtain

$$\sin(\tilde{X}) = H \sinh(1) \cos(h) + iH \cosh(1) \sin(h).$$

This algebra explains the result; it need not be implemented explicitly. Julia evaluates `sin((1 + im*h)*H)` directly, and for this input the result is correct. The end of this section explains why that cannot be taken for granted.

3. Extract a matrix of derivatives. Divide each imaginary entry by h . Since $\sin(h)/h = 1 + O(h^2)$,

$$\boxed{\frac{\text{Im} \sin(\tilde{X})}{h} = H \cosh(1) \frac{\sin(h)}{h} \approx F'(1) = H \cosh(1).}$$

Julia calculation (examples/ctse_matrix.jl).

```
using LinearAlgebra
H = [0.0 1.0; -1.0 0.0]
t0, h = 1.0, 1e-20
G = sin((t0 + im*h)*H)
value = real.(G)
derivative = imag.(G)/h
@show value derivative
```

Julia’s `sin(X)` computes the matrix sine. Applying `sin.(X)` entry by entry computes a different function: here it would give off-diagonal entries $\pm \sin(1)$ instead of $\pm \sinh(1)$ at the real input.

Compare with the analytical result. Step 2 showed that $F(t) = \sin(tH) = H \sinh(t)$ for this H . Differentiating with H fixed gives $F'(t) = H \cosh(t)$. At $t = 1$, the matrices computed above compare as follows (six decimal places):

Quantity	CTSE	Analytical
$F(1) = H \sinh(1)$	$\begin{bmatrix} 0 & 1.175201 \\ -1.175201 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 1.175201 \\ -1.175201 & 0 \end{bmatrix}$
$F'(1) = H \cosh(1)$	$\begin{bmatrix} 0 & 1.543081 \\ -1.543081 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 1.543081 \\ -1.543081 & 0 \end{bmatrix}$

The largest absolute errors among the matrix entries are 3.66×10^{-16} for the value and 1.56×10^{-16} for the derivative (references evaluated at 256-bit precision).

Where CTSE breaks. The calculation above gives the correct derivative, but only because of the structure of this example. Keep the same H and move the input in a different direction:

$$X(t) = H + tI, \quad F(t) = \sin(H + tI), \quad t_0 = 0.$$

The exact derivative is $F'(0) = I \cosh(1)$. The CTSE seed is $\tilde{X} = H + ihI$, but with $h = 10^{-20}$, `imag.(sin(H + im*h*I))/h` returns the zero matrix, without any error or warning.

Why the imaginary part is lost. For a complex matrix, Julia computes the sine from two exponentials:

$$\sin(\tilde{X}) = \frac{e^{i\tilde{X}} - e^{-i\tilde{X}}}{2i}.$$

The i in this formula is the implementation’s own complex variable. It interacts with the complex variable that carries the derivative, and the derivative information is lost.

Which matrix routines are safe. CTSE requires the complex step to be the only source of imaginary parts, with every operation the complex version of its real counterpart. Sums, products and linear solves meet this requirement. So does the matrix exponential in the thermal code, which is built from them; its CTSE results are unaffected. Routines that introduce complex numbers of their own do not: the sine and cosine above, and square roots, logarithms and other functions computed from eigenvectors or a complex Schur factorization. They fail silently, returning zeros or meaningless values.

From complex steps to dual numbers. The conflict arises because CTSE and the routine both use `i`. Dual numbers in CR form avoid it by carrying the derivative in a real block matrix, so there is no second `i` to collide with. Julia’s matrix sine can then be used as it is, even though it works through complex exponentials internally.

3 Dual numbers in CR form: a scalar input

A dual number is $a + b\varepsilon$, where ε is a new unit with $\varepsilon \neq 0$ but $\varepsilon^2 = 0$. Dual numbers add and multiply like polynomials in ε , with every ε^2 term dropped:

$$(a + b\varepsilon) + (c + d\varepsilon) = (a + c) + (b + d)\varepsilon, \quad (a + b\varepsilon)(c + d\varepsilon) = ac + (ad + bc)\varepsilon.$$

The ε part of the product follows the product rule. For a smooth function, the Taylor series in ε stops after the linear term:

$$f(a + b\varepsilon) = f(a) + f'(a) b\varepsilon.$$

Seeding $b = 1$ therefore carries the derivative in the ε part, exactly and without a step size.

Worked example: $\sin(x)$ in dual arithmetic. Seed $\tilde{x} = 1 + \varepsilon$: value 1, derivative seed 1. Expanding the sine,

$$\sin(1 + \varepsilon) = \sin(1) + \cos(1)\varepsilon - \frac{\sin(1)}{2!}\varepsilon^2 - \dots = \sin(1) + \cos(1)\varepsilon,$$

since $\varepsilon^2, \varepsilon^3, \dots$ all vanish. The real part is $f(1) = \sin(1)$ and the ε part is $f'(1) = \cos(1)$. In CTSE, $i^2 = -1$ keeps the higher-order terms, which is the source of its $O(h^2)$ error; here they vanish exactly, and nothing is divided by h .

Julia calculation (`examples/dual_scalar.jl`).

```
struct Dual <: Real
    a::Float64 # value
    b::Float64 # derivative
end
Base.sin(x::Dual) = Dual(sin(x.a), cos(x.a)*x.b)
y = sin(Dual(1.0, 1.0))
value, derivative = y.a, y.b
@show value derivative
```

Why not implement dual numbers directly? The snippet works because it teaches Julia one rule: how the sine acts on a dual number. A real code needs such a rule for every operation it uses (addition, multiplication, division and each elementary function), and every variable along the calculation must change to the dual type. Complex numbers are built into programming languages and numerical libraries, which is why CTSE needs little more than a complex input. Dual numbers are not. Library routines are the harder obstacle: Julia’s matrix exponential and sine are implemented only for the floating-point types that LAPACK supports, so even a dual type with full arithmetic cannot be passed to them.

The remedy: the CR form. Dual numbers can instead be represented by real matrices, so that existing real routines do the dual arithmetic. In the lower triangular Cauchy–Riemann (CR) convention used by this repository,

$$a + b\varepsilon \longmapsto C(a + b\varepsilon) = \begin{bmatrix} a & 0 \\ b & a \end{bmatrix}.$$

Replacing ε by $N = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}$ preserves $\varepsilon^2 = 0$, because $N^2 = 0$. Matrix addition and multiplication then reproduce dual-number arithmetic, and so does any function given by a power series, including the matrix sine. No new number type is needed: the code only has to accept real matrices.

1. Seed the same scalar function. For $f(x) = \sin(x)$ at $x = x_0$, use $x_0 + \varepsilon$, represented by

$$S = \begin{bmatrix} x_0 & 0 \\ 1 & x_0 \end{bmatrix} = x_0 I + N.$$

2. Evaluate the sine. Start with the powers of S . Squaring gives

$$S^2 = \begin{bmatrix} x_0 & 0 \\ 1 & x_0 \end{bmatrix} \begin{bmatrix} x_0 & 0 \\ 1 & x_0 \end{bmatrix} = \begin{bmatrix} x_0^2 & 0 \\ 2x_0 & x_0^2 \end{bmatrix},$$

and continuing in this way, for $k \geq 1$,

$$S^k = \begin{bmatrix} x_0^k & 0 \\ k x_0^{k-1} & x_0^k \end{bmatrix}.$$

The lower-left entry is the derivative of x_0^k : matrix multiplication applies the product rule, exactly as dual multiplication did. Now substitute these powers into the sine series:

$$\begin{aligned} Y = \sin(S) &= S - \frac{S^3}{3!} + \frac{S^5}{5!} - \cdots \\ &= \begin{bmatrix} x_0 - \frac{x_0^3}{3!} + \frac{x_0^5}{5!} - \cdots & 0 \\ 1 - \frac{3x_0^2}{3!} + \frac{5x_0^4}{5!} - \cdots & x_0 - \frac{x_0^3}{3!} + \frac{x_0^5}{5!} - \cdots \end{bmatrix} = \begin{bmatrix} \sin(x_0) & 0 \\ \cos(x_0) & \sin(x_0) \end{bmatrix}. \end{aligned}$$

The diagonal entries are the sine series. In the lower-left entry, the factors $3, 5, \dots$ simplify against the factorials, leaving the cosine series. Julia’s matrix sine evaluates `sin(S)` directly; no dual-number type or Taylor expansion has to be implemented.

3. Read the first column.

$$f(x_0) = Y_{11} = \sin(x_0), \quad f'(x_0) = Y_{21} = \cos(x_0).$$

Now substitute $x_0 = 1$ to obtain $f(1) = \sin(1)$ and $f'(1) = \cos(1)$.

Julia calculation (`examples/dual_cr_scalar.jl`).

```
using LinearAlgebra
x0 = 1.0
S = [x0 0.0; 1.0 x0]
Y = sin(S)
value = Y[1, 1]
derivative = Y[2, 1]
@show value derivative
```

Compare with the analytical result. Direct differentiation of $f(x) = \sin(x)$ gives $f'(x) = \cos(x)$. At $x = 1$, the entries extracted from Y compare as follows:

Quantity	Dual CR	Analytical
$f(1) = \sin(1)$	0.841470984808	0.841470984808
$f'(1) = \cos(1)$	0.540302305868	0.540302305868

Both computed entries equal the analytical values rounded to double precision (references evaluated at 256-bit precision). The derivative is exact in dual arithmetic; the numerical matrix sine still introduces floating-point error. There is no h to choose or divide by. These are the same numbers CTSE produced in Section 1: with $h = 10^{-20}$, its $O(h^2)$ error is about 10^{-41} , far below rounding. For first derivatives, the two methods differ in robustness rather than accuracy.

4 Dual numbers in CR form: a matrix input

Return to $\mathcal{F}(X) = \sin(X)$ with $X(t) = tH$ and $H = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$ at $t_0 = 1$. The physical input is a 2×2 matrix. Its CR representation is a 4×4 real matrix containing both value and derivative information.

1. Seed the matrix input. First seed the scalar timestep, just as in the scalar example:

$$t_0 \mapsto T = t_0 I_2 + N = \begin{bmatrix} t_0 & 0 \\ 1 & t_0 \end{bmatrix}, \quad N = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}.$$

For $X(t) = tH$, form its CR representation by multiplying each entry of T by H :

$$\mathcal{M} = T \otimes H = \begin{bmatrix} t_0 H & 0 \\ H & t_0 H \end{bmatrix}.$$

This construction is the Kronecker product, written `kron(T, H)` in Julia. Now substitute $t_0 = 1$:

$$\boxed{\mathcal{M} = \begin{bmatrix} H & 0 \\ H & H \end{bmatrix}} = \left[\begin{array}{cc|cc} 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ \hline 0 & 1 & 0 & 1 \\ -1 & 0 & -1 & 0 \end{array} \right].$$

2. Evaluate one real matrix sine. First square $\mathcal{M}$ using ordinary block multiplication:

$$\mathcal{M}^2 = \begin{bmatrix} H & 0 \\ H & H \end{bmatrix} \begin{bmatrix} H & 0 \\ H & H \end{bmatrix} = \begin{bmatrix} H^2 & 0 \\ HH + HH & H^2 \end{bmatrix} = \begin{bmatrix} H^2 & 0 \\ 2H^2 & H^2 \end{bmatrix}.$$

Multiplying once more gives

$$\mathcal{M}^3 = \mathcal{M}^2 \mathcal{M} = \begin{bmatrix} H^3 & 0 \\ 2H^3 + H^3 & H^3 \end{bmatrix} = \begin{bmatrix} H^3 & 0 \\ 3H^3 & H^3 \end{bmatrix}.$$

Continuing in this way gives, for $k \geq 1$,

$$\mathcal{M}^k = \begin{bmatrix} H^k & 0 \\ kH^k & H^k \end{bmatrix}.$$

Now substitute these powers into the matrix sine series:

$$\begin{aligned}\sin(\mathcal{M}) &= \mathcal{M} - \frac{\mathcal{M}^3}{3!} + \frac{\mathcal{M}^5}{5!} - \dots \\ &= \begin{bmatrix} H - \frac{H^3}{3!} + \frac{H^5}{5!} - \dots & 0 \\ H - \frac{3H^3}{3!} + \frac{5H^5}{5!} - \dots & H - \frac{H^3}{3!} + \frac{H^5}{5!} - \dots \end{bmatrix}.\end{aligned}$$

Each diagonal block is the series for $\sin(H)$. In the lower-left block, the factors 3, 5, ... simplify against the factorials:

$$H - \frac{3H^3}{3!} + \frac{5H^5}{5!} - \dots = H \left(I - \frac{H^2}{2!} + \frac{H^4}{4!} - \dots \right) = H \cos(H).$$

For our matrix H , $H^2 = -I$, so $\sin(H) = H \sinh(1)$ and $\cos(H) = I \cosh(1)$. Writing $s = \sinh(1)$ and $c = \cosh(1)$, the result is

$$\sin(\mathcal{M}) = \begin{bmatrix} \sin(H) & 0 \\ H \cos(H) & \sin(H) \end{bmatrix} = \left[\begin{array}{cc|cc} 0 & s & 0 & 0 \\ -s & 0 & 0 & 0 \\ \hline 0 & c & 0 & s \\ -c & 0 & -s & 0 \end{array} \right].$$

As in the scalar example, a matrix sine routine evaluates $\sin(\mathcal{M})$ directly; the series above explains that result.

3. Read the first block column.

$$F(1) = \sin(\mathcal{M})_{1:2, 1:2}, \quad F'(1) = \sin(\mathcal{M})_{3:4, 1:2}.$$

Julia calculation (examples/dual_cr_matrix.jl).

```
using LinearAlgebra
H = [0.0 1.0; -1.0 0.0]
T = [1.0 0.0; 1.0 1.0]
G = sin(kron(T, H))
value = G[1:2, 1:2]
derivative = G[3:4, 1:2]
@show value derivative
```

The lower-left block carries the chain rule through the matrix sine. This is forward-mode differentiation encoded in real block arithmetic.

Compare with the analytical result. As in Section 2, $F(t) = \sin(tH) = H \sinh(t)$ for this H , so $F'(t) = H \cosh(t)$. At $t = 1$, the extracted blocks compare as follows (six decimal places):

Quantity	Dual CR	Analytical
$F(1) = H \sinh(1)$	$\begin{bmatrix} 0 & 1.175201 \\ -1.175201 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 1.175201 \\ -1.175201 & 0 \end{bmatrix}$
$F'(1) = H \cosh(1)$	$\begin{bmatrix} 0 & 1.543081 \\ -1.543081 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 1.543081 \\ -1.543081 & 0 \end{bmatrix}$

The largest absolute errors among the matrix entries are 3.66×10^{-16} for the value and 3.78×10^{-16} for the derivative (references evaluated at 256-bit precision).

Beyond first derivatives. Dual numbers are the first-order case of order-truncated imaginary (OTI) numbers. The same CR construction carries derivatives of any order, with respect to any number of parameters, still in one evaluation of a real matrix function.

Choosing between CTSE and dual CR

For first derivatives, both methods reach full double precision. CTSE needs a step h and a routine that is safe for CTSE (Section 2), and in its basic form it gives only first derivatives. Dual CR has no step, works with ordinary real routines, and extends to any order and any number of parameters (end of Section 4). Its price is size: the matrix doubles in dimension, and a dense matrix function then costs roughly twice as much as with CTSE. Routines that diagonalize their input do not suit dual CR: the CR matrix cannot be diagonalized, and such routines lose about half the digits.

Connecting the examples to this repository

The examples use sine; the thermal model uses a matrix exponential. The same seeding and extraction steps apply to both functions. The model uses

$$H = \begin{bmatrix} A & B & I_n \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad z = \begin{bmatrix} x \\ u \\ e \end{bmatrix}, \quad R(t) = [\exp(tH)z]_{1:n}.$$

Here A describes thermal dynamics, B the input coupling, x the initial temperatures, u the fixed input, and e the fixed forcing vector. The first n entries give the temperatures after time t ; the remaining augmented components keep the input and forcing constant.

CTSE in the code. `ctse_response` in `src/SensitivityComparison.jl` evaluates `response` at $t + ih$, then extracts

$$R(t) \approx \operatorname{Re} R(t + ih), \quad R'(t) \approx \operatorname{Im} R(t + ih)/h.$$

This follows the matrix CTSE example with the exponential in place of sine, then multiplication and selection of the temperature entries.

Dual numbers in the code. With `order=1`, `oti_response` in `src/OTISensitivity.jl` constructs `oti_matrix([t, 1.0])`, which is exactly $T = \begin{bmatrix} t & 0 \\ 1 & t \end{bmatrix}$. It exponentiates `kron(T, H)`. If m is the size of H and G is that exponential, the extraction is

$$R(t) = G_{1:n, 1:m} z, \quad R'(t) = G_{m+1:m+n, 1:m} z.$$

The returned `derivatives[:, 1]` contains temperatures and `derivatives[:, 2]` their timestep derivatives.

Results for the thermal model. The example model has ten voxels in a row. All start at 300 K except voxel 10, at 900 K, and voxel 1 receives a constant heat input; there is no forcing. With a timestep of $t = 50$ s, CTSE evaluates one complex 21×21 matrix exponential and first-order CR one real 42×42 exponential. The table compares both with a reference that evaluates $\exp(tH)z$ and its derivative $H \exp(tH)z$ in 256-bit arithmetic.

Julia calculation (examples/thermal.jl).

```
include(joinpath(@_DIR_, "..", "src", "SensitivityComparison.jl"))
using .SensitivityComparison
(; A, B, e, x, u, dt) = thermal_example() # ten voxels, t = 50 s

# CTSE: evaluate the response at t + ih.
ctse = ctse_response(A, B, e, x, u, dt; h=1e-20)
# Dual CR: exponentiate kron(T, H) with T = [t 0; 1 t].
cr = oti_response(A, B, e, x, u, dt; order=1)
@show ctse.value ctse.derivative cr.derivatives[:, 2]
```

Voxel	$R_i(t)$ [K]	$R'_i(t)$ [K/s]	CTSE relative error		Dual CR relative error	
			$R_i(t)$	$R'_i(t)$	$R_i(t)$	$R'_i(t)$
1	586.661	3.8615	2.3×10^{-14}	1.6×10^{-14}	1.4×10^{-14}	2.2×10^{-15}
2	433.198	2.9770	1.4×10^{-14}	2.1×10^{-14}	8.8×10^{-15}	9.2×10^{-15}
3	354.161	1.8339	3.7×10^{-16}	3.5×10^{-14}	3.6×10^{-14}	2.1×10^{-14}
4	320.971	1.0108	4.3×10^{-16}	1.9×10^{-14}	5.6×10^{-14}	4.9×10^{-14}
5	313.049	0.7028	6.3×10^{-15}	6.0×10^{-14}	5.8×10^{-14}	3.5×10^{-14}
6	322.698	0.8112	6.2×10^{-15}	2.5×10^{-14}	5.5×10^{-14}	1.9×10^{-14}
7	352.627	0.9912	8.3×10^{-15}	3.2×10^{-14}	4.5×10^{-14}	7.5×10^{-15}
8	407.336	0.6319	2.4×10^{-15}	4.7×10^{-14}	3.1×10^{-14}	2.5×10^{-15}
9	477.844	-0.6758	2.6×10^{-15}	2.0×10^{-14}	8.2×10^{-15}	1.4×10^{-13}
10	531.455	-2.1445	1.5×10^{-15}	1.0×10^{-15}	3.4×10^{-15}	4.9×10^{-14}

The largest relative errors over the ten voxels are 2.30×10^{-14} and 5.97×10^{-14} for CTSE, and 5.83×10^{-14} and 1.40×10^{-13} for dual CR, for the temperatures and derivatives respectively (reference evaluated at 256-bit precision). Voxel 1, heated by the input, warms fastest; voxels 9 and 10, which start hot, are cooling.

Reproduce the examples. Each Julia calculation above is a script in `examples/`, shown verbatim apart from a one-line header. From the repository root, run one of them, or the verification script that runs them all:

```
julia --project=. examples/thermal.jl
julia --project=. docs/verify_ctse_dual_cr_examples.jl
```

The verification script checks all four examples against their explicit formulas, confirms the CTSE failure at the end of Section 2, and checks the first derivatives of the thermal response against the ODE. It also regenerates the five comparison tables, including the thermal results, from the computed results.